

EMOTION IN DEBUGGING

You walk up to one of your students...

The student is debugging...

You start to provide them with some support...

How do you factor in what students are conveying emotionally, how they're feeling? Free-write for a few minutes. Share out.

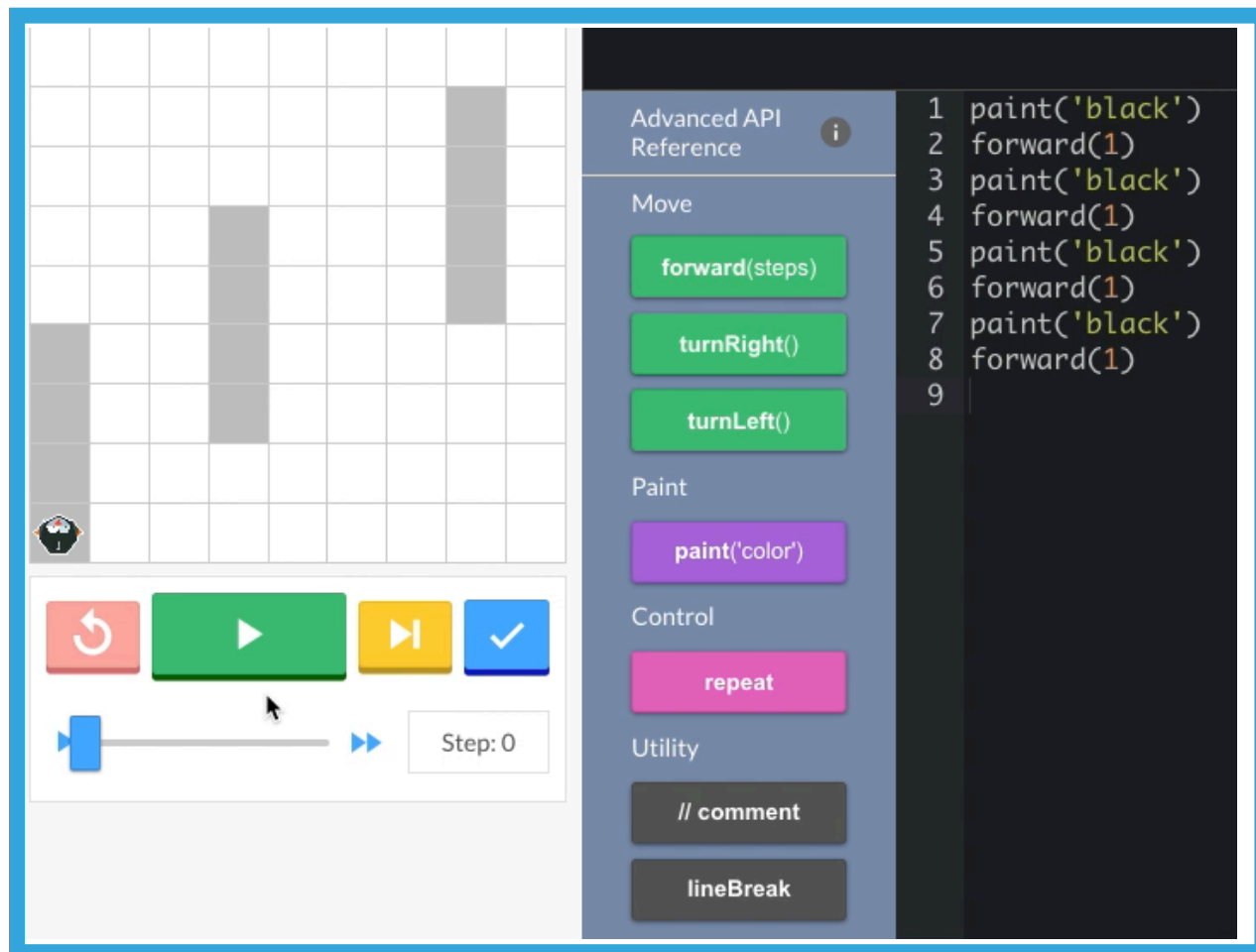
- ▶ Browse around this visualization:
<https://tinyurl.com/debugemotion>

- ▶ Browse around this visualization:
<https://tinyurl.com/debugemotion>

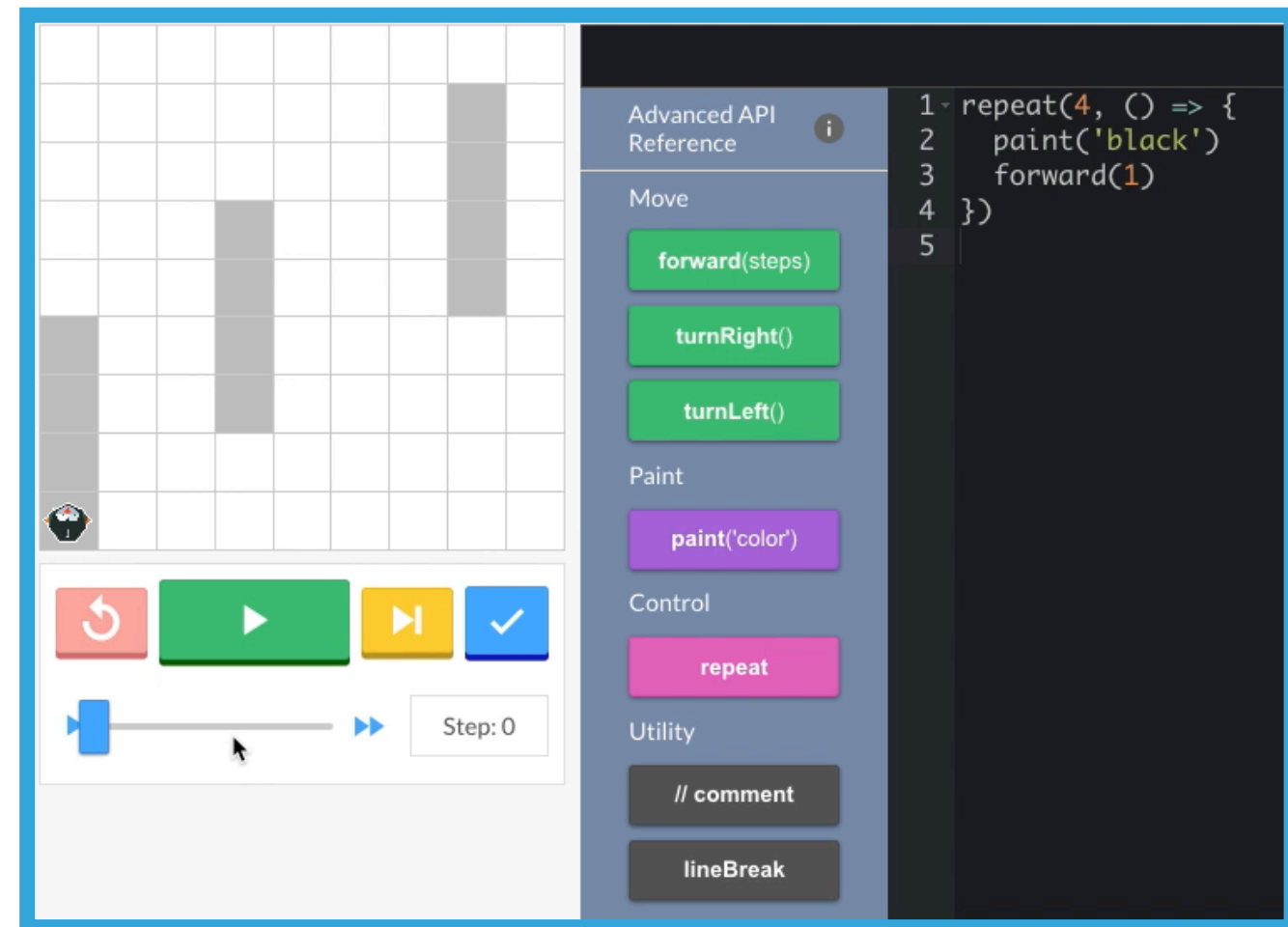
How do we grapple with this wide range of emotion as CS educators? What stories come to mind for you?

REFACTORING

SEQUENCE OF COMMANDS



LOOP (FOR-LOOP OR REPEAT LOOP)

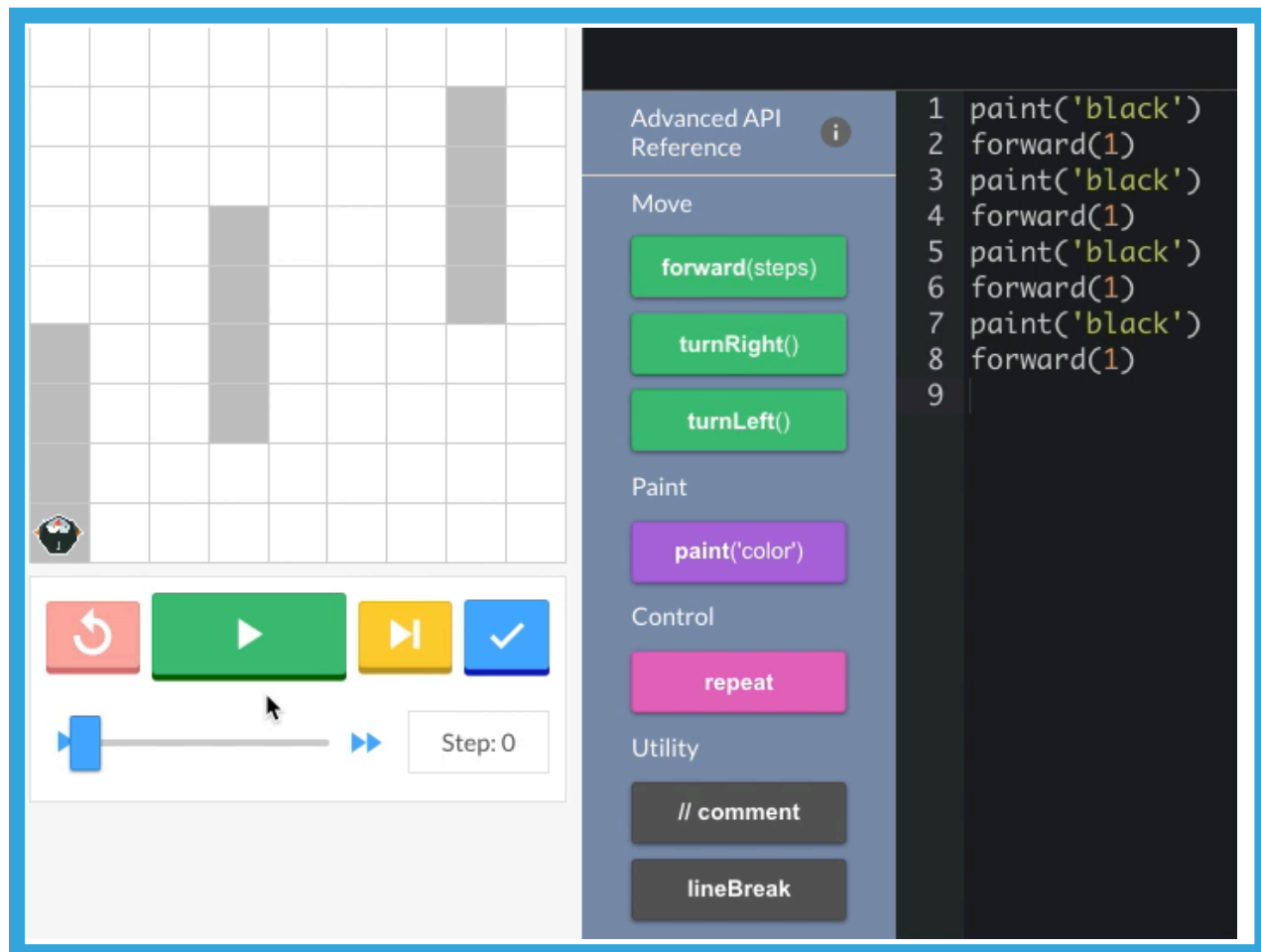


REFACTORING

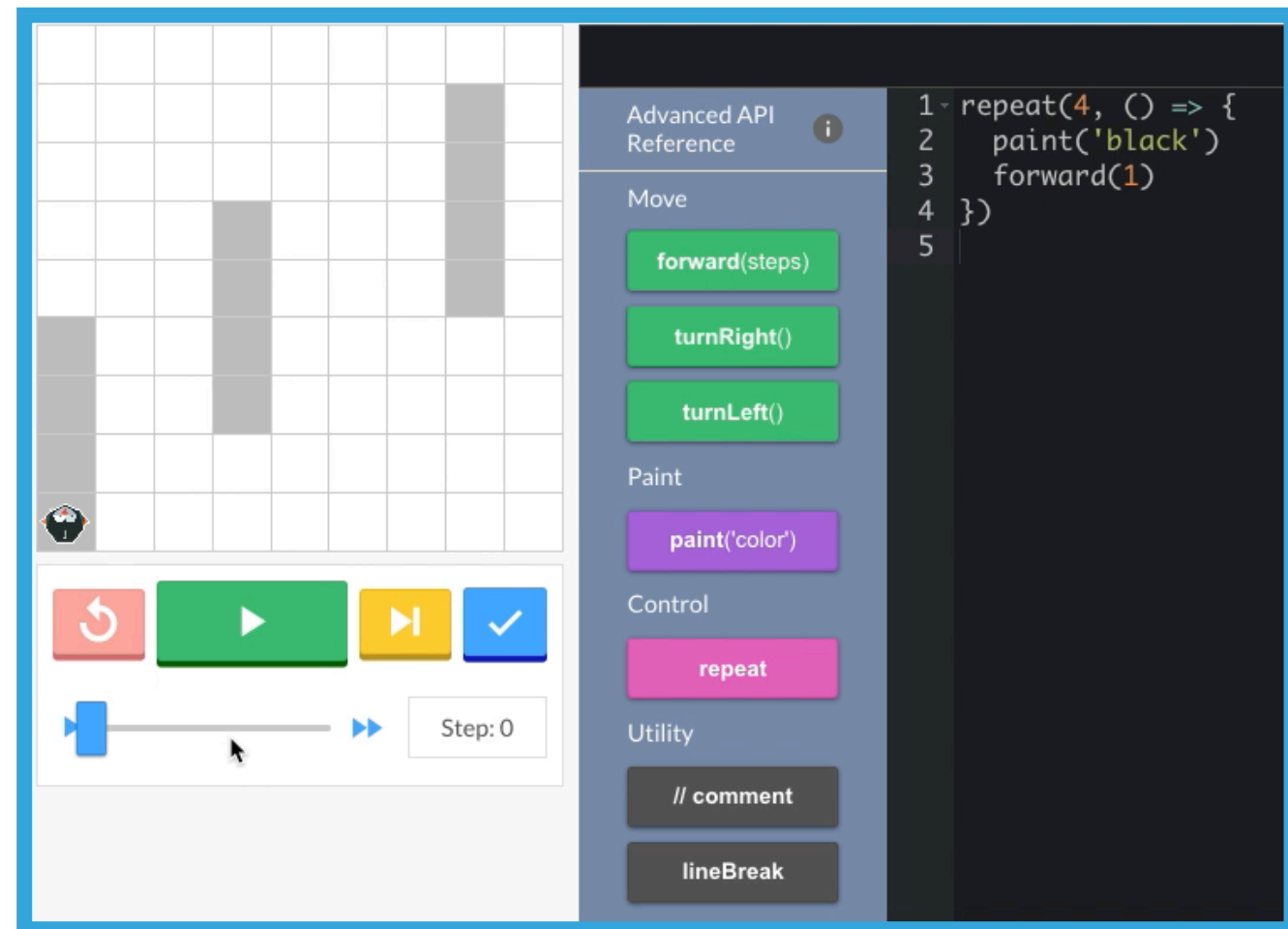
"Refactoring is the process of changing a software system in such a way that it **does not alter the external behavior of the code** yet **improves its internal structure**. It is a disciplined way to clean up code that minimizes the chances of introducing bugs. In essence when you refactor you are improving the design of the code after it has been written." (p. 9)

REFACTORING

SEQUENCE OF COMMANDS



LOOP (FOR-LOOP OR REPEAT LOOP)



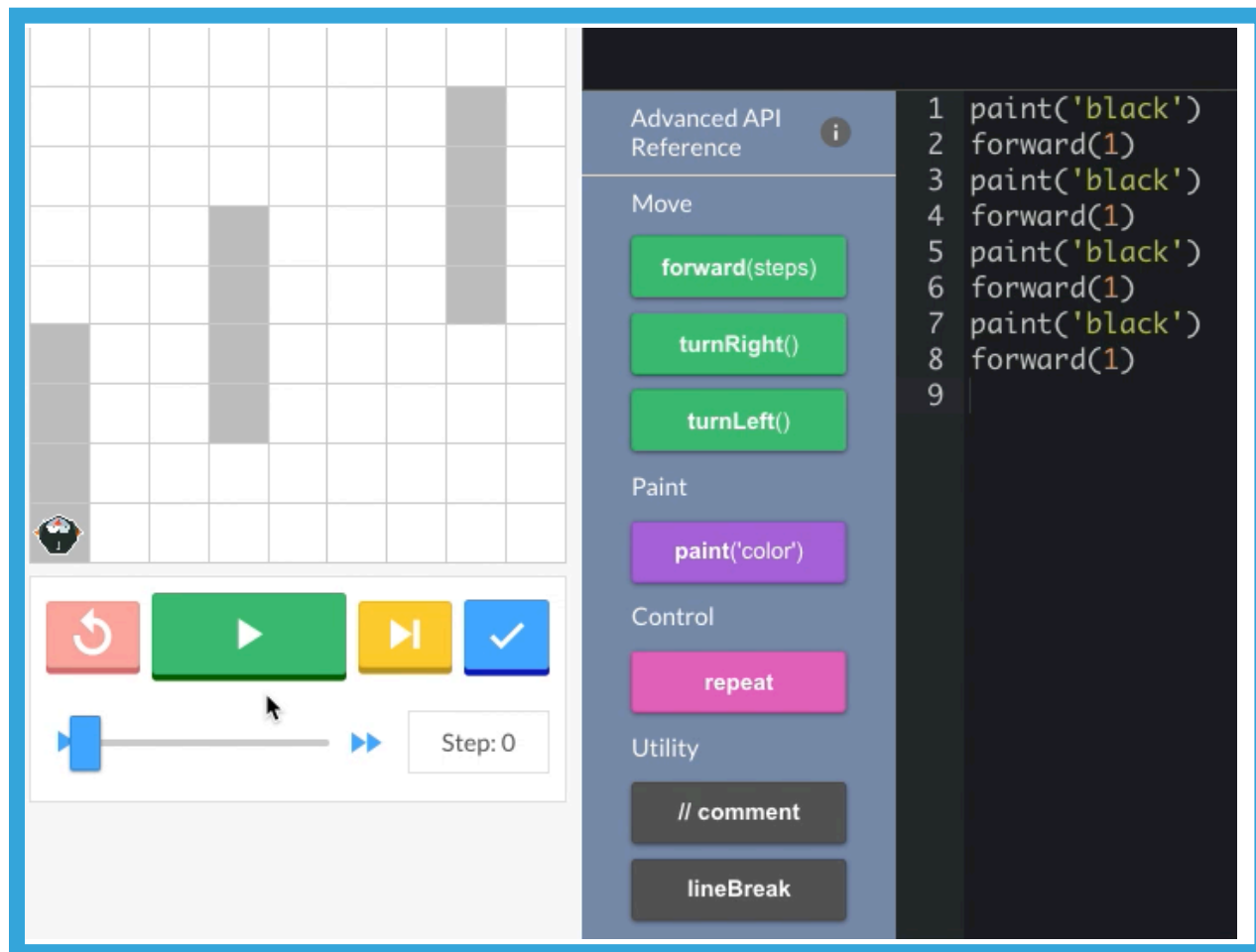
REFACTORING



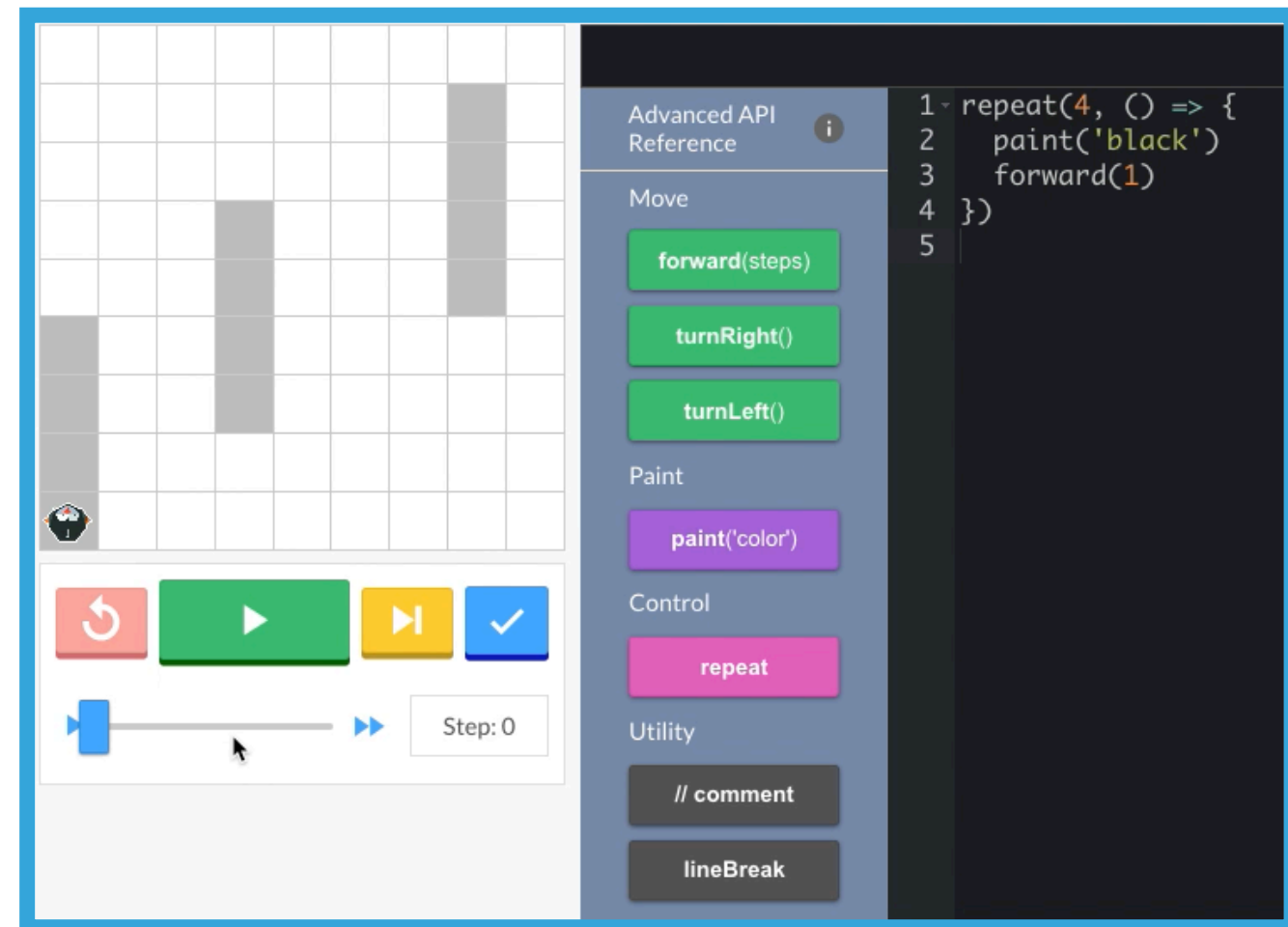
"Refactoring is the process of changing a software system in such a way that it **does not alter the external behavior of the code** yet **improves its internal structure**. It is a disciplined way to clean up code that minimizes the chances of introducing bugs. In essence when you refactor you are improving the design of the code after it has been written." (p. 9)

REFACTORING

SEQUENCE OF COMMANDS



LOOP (FOR-LOOP OR REPEAT LOOP)



REFACTORING

"Refactoring is the process of changing a software system in such a way that it **does not alter the external behavior of the code** yet **improves its internal structure**. It is a disciplined way to clean up code that minimizes the chances of introducing bugs. In essence when you refactor you are improving the design of the code after it has been written." (p. 9)

REFACTORING



ALMA: Delete everything. (1.5) Look, I'm deleting everything.

VERA: (Gasps) Noo! ... Do you regret your work?

ALMA: No, I do not regret it.

VERA: Did you make a big mistake?

ALMA: No. (starts running code)

VERA: Whaat? Why did you turn right—

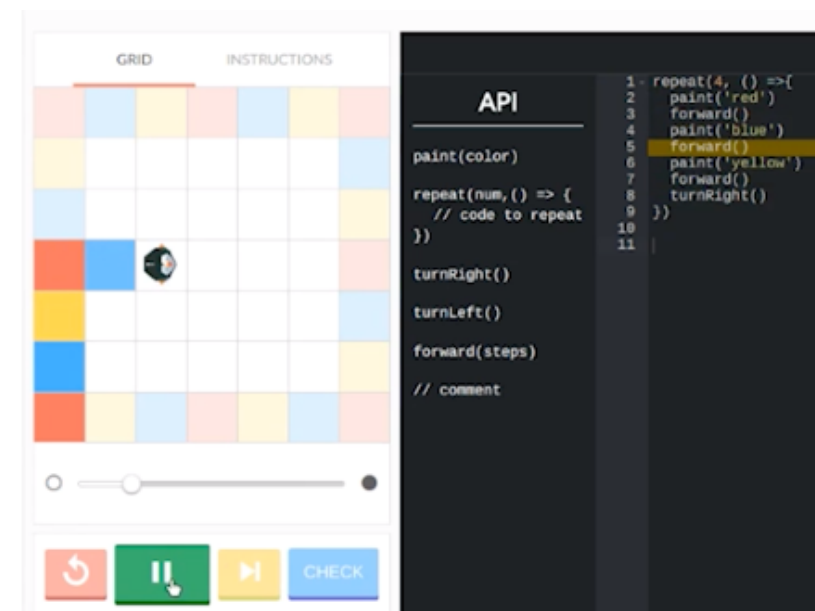
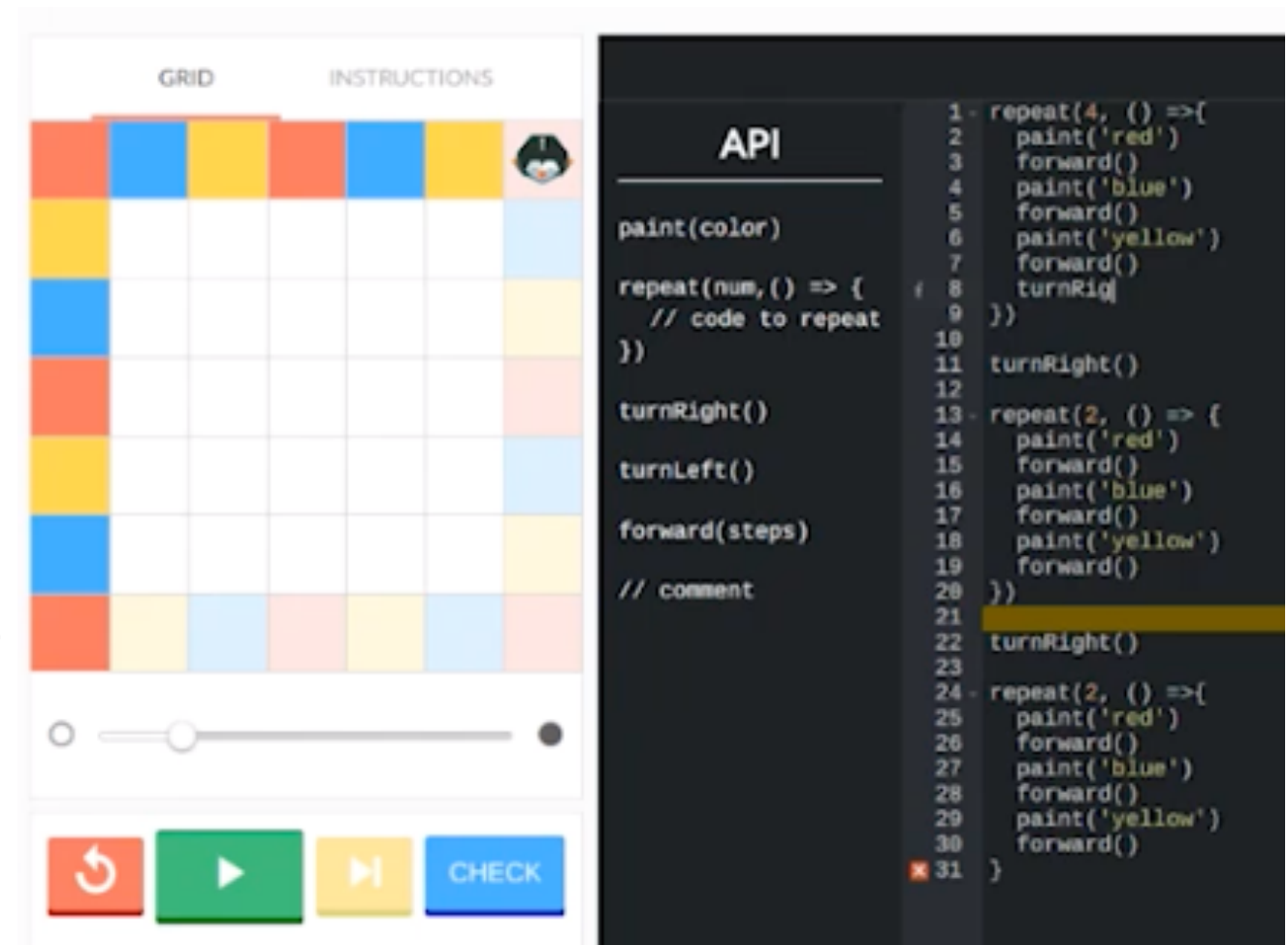
ALMA: OOOOOPS!

VERA: (laughing) Why did you turn right?

ALMA: I messed up!

VERA: Whaat!

ALMA: I'm not supposed to do that, now I regret it.



REFACTORING

- ▶ Papert (1980) sounded an alarm more than 40 years ago. Several anecdotes problematized when to ask students to refactor:
 - ▶ “Deborah decided to restrict her Turtle commands, creating a microworld within the microworld of Turtle commands. She allowed herself only one turning command: RIGHT 30. To turn the Turtle through 90 degrees, she would repeat RIGHT 30 three times and would obtain the effect of LEFT 30 by repeating it eleven times. To an onlooker it might seem tedious to obtain simple effects in such complicated ways. But for Deborah it was exciting to be able to construct her own microworld and to discover how much she could do within its rigid constraints. She no longer asked permission to explore. And one day, when the teacher offered to show her a ‘simpler way’ to achieve an effect, she listened patiently and said, ‘I don’t think I’ll do it that way.’”
- ▶ Turkle and Papert (1990) later argued that programming education environments limit **epistemological pluralism**, defined as “accepting the validity of multiple ways of knowing and thinking”

REFACTORING

- ▶ Friction, failure, breakdown prompt individuals to dedicate effort and time to reflect on and consider adapting their process – not moments judged as successful (Kapur & Kinzer, 2009; Koschmann et al., Posner et al., 1982; 1998; Reason, 1990; Weiner, 1985)
- ▶ Efficiency in problem solving tied to exclusionary practices along racial and gendered lines in undergraduate mathematics courses (Leyva et al., 2021)
- ▶ Backdrop of recent scholarship on computing learning spaces that documents the resistance students face to asserting their agency during learning (Ryoo et al., 2020; Vakil, 2020)

REFACTORING

- ▶ Friction, failure, breakdown prompt individuals to dedicate effort and time to reflect on and consider adapting their process – not moments judged as successful (Kapur & Kinzer, 2009; Koschmann et al., Posner et al., 1982; 1998; Reason, 1990; Weiner, 1985)
- ▶ Efficiency in problem solving tied to exclusionary practices along racial and gendered lines in undergraduate mathematics courses (Leyva et al., 2021)
- ▶ Backdrop of recent scholarship on computing learning spaces that documents the resistance students face to asserting their agency during learning (Ryoo et al., 2020; Vakil, 2020)

REFACTORING

- ▶ Friction, failure, breakdown prompt individuals to dedicate effort and time to reflect on and consider adapting their process – not moments judged as successful (Kapur & Kinzer, 2009; Koschmann et al., Posner et al., 1982; 1998; Reason, 1990; Weiner, 1985)
- ▶ Efficiency in problem solving tied to exclusionary practices along racial and gendered lines in undergraduate mathematics courses (Leyva et al., 2021)
- ▶ Backdrop of recent scholarship on computing learning spaces that documents the resistance students face to asserting their agency during learning (Ryoo et al., 2020; Vakil, 2020)

Are there situations in your teaching when you position students' work as flawed when they might not (reasonably) see it that same way? How do you approach these moments? What emotions are in play? **Free-write for a few minutes.**

INTRODUCING (SOCIOCOMPUTATIONAL) NORMS

- ▶ One instructor, Meg, stands at the front of the room and introduces the activity to the whole class. Meg introduces three norms:
 - ▶ (1) **be resourceful** with what you know ("This is to show me and show yourself how much you learned" and "Use what you know.")
 - ▶ (2) **embrace difficulty**/struggle, don't be afraid of it (Call and response chant: "Bring it! Bring it!")
 - ▶ (3) **work quickly** and efficiently (10 minutes for activity, students push back, "Okay I'll give you give 5 minutes")

LAUNCHING REFACTORING – HACKERS AND THE LOOP

01 Ash: [HHh ((gasp)) I fihhgured ouht a way to che::at
02 [—

03 *(((Software developers and Meg look at Ash))*
04 Ash: [I'm- I'm a hackER (.) I'm a hacker now
05 *(((Developers smile while alternating looks at each other and Ash))*
06 *((Meg and the developers approach the table))*
07 Dex: I gotta see what this (could/would) be
08 Joe: Hhh ((laugh))
09 Meg: [What's- what's your hack
10 *(((Meg leans down to look at Ash's screen))*
11 *((Ash highlights a line of code))*
12 *((Meg flips her hand open))*
13 Meg: [#What's your hack?#
14 [—
15 *(((Ash clicks command + C))*
16 Dex: Ah:h (he found) copy and paste () a hack
17 Meg: #Oh:h [okay#
18 *(((Ash pastes the line of code))*
19 *((Ash raises his arms and bobs his head again and the developers laugh))*
20 Ash: [Copy and paste is the WORLD best
21 *(((Ash deletes the line he pasted))*
22 Zoa: You (told) us nothing new
23 *(((The developers start to wander away))*
24 Ash: Yeah [YOU (COULD/CAN) ALL DO IT NOW (.)
25 COPY AND PASTE (.) THE FASTEST WAY TO FINISH
26 *(((Ash raises his arms again))*



Ash, left-most, leans back, looks up, raises his arms, and bobs his head side to side as Meg and the developers (black hair) walk around the room.



All three developers and Meg huddle around Ash.

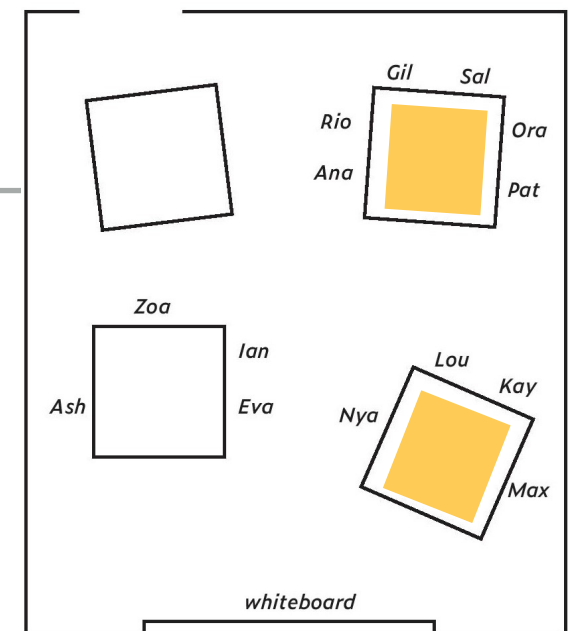
LAUNCHING REFACTORING – HACKERS AND THE LOOP

27 Meg: Hey if you guys are copying and pasting a lot
28 think about [maybe using: ↴a loop (1.3)]
29 [—————
30 Meg: if you're repeating a lot of the same [code]
31 Ash: [>Nayah<]
32 Ash: [I like copy and pas::te so (.) SO much better
33 *(((Ash leans back, shakes his head, and lifts his hands off the computer))*
34 *(((Eva looks at Meg and laughs))*
35 Zoa: Less typing [(mmm/y'know/though)
36 Meg: [But here's
37 Meg: [the API for a loop (.) so (.)
38 *(((Meg points at repeat loop in API while Ash runs his code))*
39 Ash: Yep (.) I (d') want it (.)
40 Meg: Mhmm (.) [so keep doing it how] you're doing
41 *(((Meg points to Ash's screen)))*
42 it and maybe you'll notice [a pattern] later
43 Ash: [Right]
44 (1.4)
45 Ash: [tu::rn (1.2) (do- do::o) the copy and paste better (.)
46 Much faster
47 *(((Meg walks away from Ash to look at Eva's screen))*



Meg makes a looping gesture.

REFACTORING DISCUSSIONS SPREAD



- ▶ Later, at the front right table, Meg notices one student coding with a repeat loop (Max) and another student hard-coding a sequence of commands (Kay). Meg remarks to the whole class, "Interesting, I see some people using loops, and some people are not," before gesturing with open palms rising and falling like a weighted scale moving up and down
 - ▶ In using the phrase "some people are not," Meg does not provide a name for these other approaches, and instead describes them relative to the absence of the loop
- ▶ At the back table, Ben (instructor) describes how Sal is using the copy-and-paste approach, and asks, "Is that the fastest strategy, Sal?" Sal says, "Yeah," and Ben mirrors Meg's comment to Ash: "If you're doing the same thing over and over again, is there a better way to do it?" Sal pushes back: "Yeah it (copy and paste) makes things faster so you won't have to type it."
 - ▶ Two sociocomputational norms of efficiency: (1) time and effort to complete the challenge and (2) recurring events create a pattern that could be coded with fewer commands

FOCAL CLIP

b.



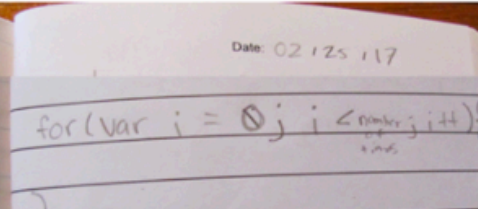
Kay does a head shake.

c.



Lou and Kay look up from their screens and toward Nya.

d.

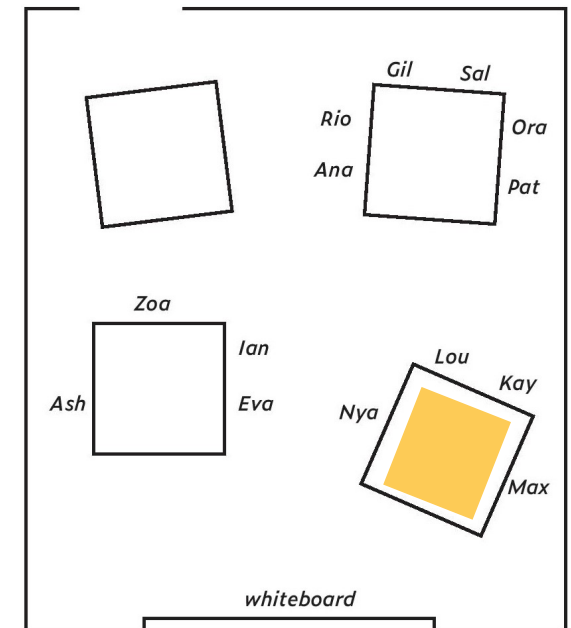


```
Date: 02/25/17  
for (var i = 0; i < number; i++) {  
    times  
}
```

Date: 02/25/17

```
for (var i = 0 ; i < number  
of  
times ; i ++ ) {  
  
}
```

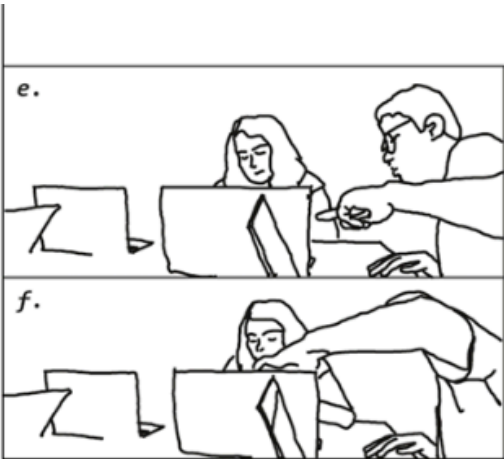
She is looking for the for-loop syntax she had written 5 months earlier.



FOCAL CLIP

29 Nya: No[:: [you could use that
30 Max: [That is
31 *(((Lou looks at Nya))*

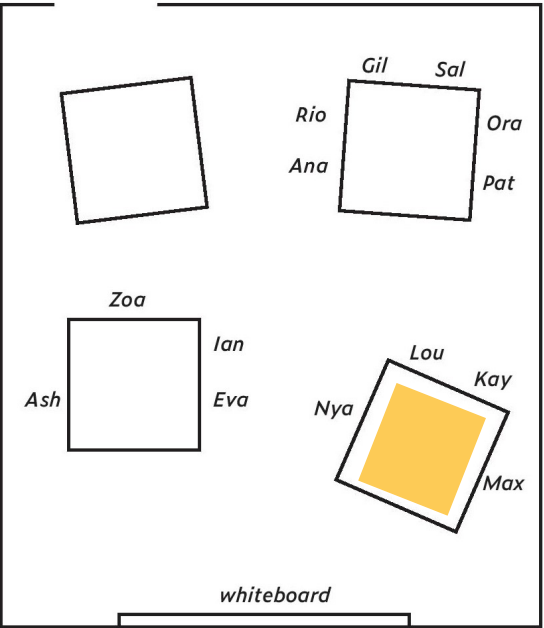
32 Kay: Ah yeah
33 *(((Kay presses her lips together))*
34 Kay: Ttch
35 *(((Lou glances toward Kay))*
36 Lou: I think I might actually do that
37 BU::[UT (.) I ↑already
38 *(((Kay looks toward Lou, then back to her screen))*
39 started so [ah it's > too late now<
40 *(((Kay looks at Lou's screen))*
41 Kay: [I already started it too and
42 [_____
43 I'm already DOne with it [so ()
44 *(((Lou coughs loudly))*



Max points to Kay's API (Application Programming Index).



Kay throws her hand up briefly and shakes her head.



MISALIGNED STANCES ON REFACTORING

01 *((Nya raises her hand to move on to the next challenge, and*
 02 *Meg asks to check her code))*
 03 *((Meg leans down to look at Nya's screen over her shoulder))* — — — —
 04 Meg: Nic::e
 05 Nya: hhh *((chuckling))*
 06 Meg: hhh *((chuckling))* ESo you [used the loopE
 07 *(((Kay looks toward Nya and Meg)))*

08

09 Nya: Yeah
 10 Meg: Oh you used ↑TWO loops
 11 Nya: [Yeah >I wanted to find out< if I::
 12 cou::ld ↑repEA:T
 13 *(((Kay looks back and forth from her screen to Nya and Meg)))*
 14 Nya: [like a loop ↑inSI:de a] loop?
 15 Meg: [Mhm]
 16 *(((Kay continues looking at Nya and Meg)))*
 17 Meg: Mhm[HMm

18 [—————

19 Max: [Oh yeAH
 20 *(((Max looks up from his screen toward Nya and points upward briefly)))*
 21 Nya: >(but then) I [didn't [want to check out<
 22 *(((Nya flips her hands palm up and then looks at Meg)))*
 23 *(((Kay looks toward Nya and Meg)))*
 24 *(((Lou looks at Max and then Nya)))*
 25
 26 Nya: like
 27 Meg: [HHHhhH *((laughing))*
 28 *(((Meg smiles widely at Nya and stands up slightly)))*
 29 *(((Nya smiles and looks at Meg)))*
 30 *(((Kay continues looking toward Nya and Meg)))*

a. *Nya's code, containing two loops.*

```
repeat(4, ()=>{
  paint('pink')
  forward(4)
  turnRight()
})

forward(1)
turnRight()
forward(1)

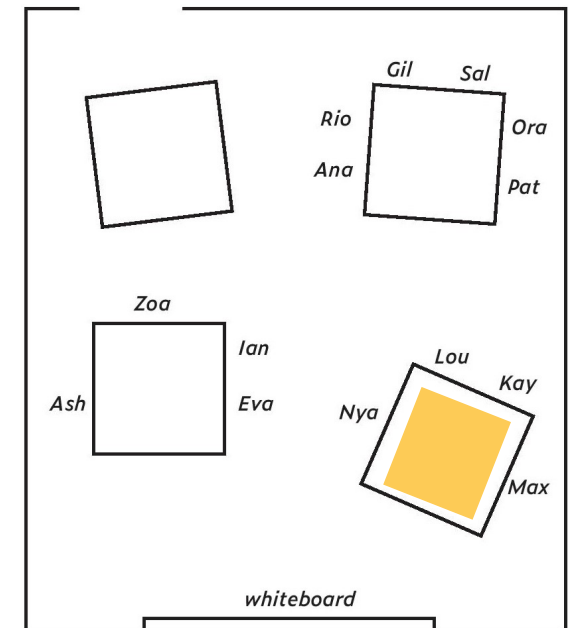
repeat(4, ()=>{
  paint('deepOrange')
  forward(1)
  paint('deepOrange')
  forward(1)
  paint('deepOrange')
  turnLeft()
})
```



Lou looks over and cranes to see Nya's screen.



Meg (black hair) smiles and turns from Nya's screen to look at Nya.



MISALIGNED STANCES ON REFACTORING

31 Nya: [()]
32 Max: [↑CA_n [yo::u
33 *(((Max continues looking toward Meg and Nya))*
34 *(((Meg and Nya look at Max))*
35 Max: do a [loop inside a [loop (.)
36 *(((Kay looks at Max))*
37 *(((Max points at his screen))*
38 *(((Nya looks at Meg))*
39 *((Meg nods))*
40 Meg: [Yes
41 Nya: [Yeah [can you]
42 Max: [cuz that's-]
43 *(((Kay looks upward then at Meg)))*
44 Max: [I'm trying to
45 Meg: [Yeah you can
46 *((Meg glances at Nya then looks again at Max))*
47 Meg: [Mhmhh
48 Kay: You [ca::n but it's [too confusing]
49 [_____]
50 *(((Meg glances at Kay then looks at Nya's screen))*
51 Nya: [>I wanna make myself look<]
52 coo::l
53 *(((Kay looks back and forth between Nya and Meg))*
54 *(((Kay looks at Nya while smiling widely))*
55 Meg: [You should- you guys should [try
56 *(((Meg looks at Nya's screen and then at Max))*
57 [f. _____]
58 *(((Kay looks up at Meg))*
59 Max: Yeah
60 Meg: If [you- [if you're curious about thAT (.)
61 [g. _____]
62 *(((Kay nods her head once and frowns briefly))*
63 Meg: Definitely go for it
64 *((Meg walks away))*



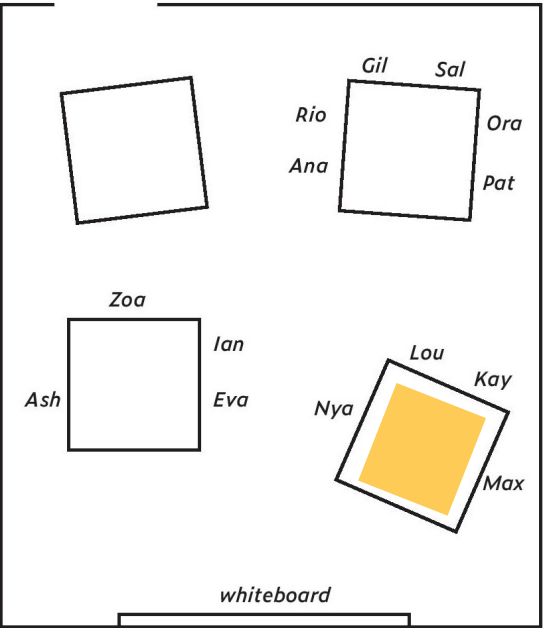
Nya lifts her palms up and shakes her head.



Kay touches her ear while talking.



(f) Meg touches her neck.
(g) Meg points to the table with a splayed hand.



CONTINUED EMPHASIS ON REFACTORING USING LOOPS

01 ((David has been helping Kay use a loop and is now silently watching Max code.))
02 ((Meg is looking on from afar))
03 Nya: [There we go! (.) °Hhhh° ((gasping softly))
04 [(((Nya leans back from her screen)))



Nya lifts her hands off her keyboard ...

06 ((...and turns to look at Meg))
07 Nya: °I [feel so cool°



Meg (black hair) walks over and leans to look at Nya's screen.

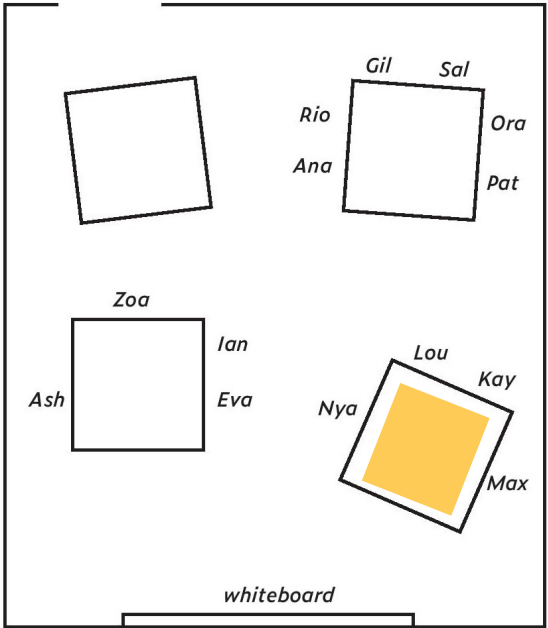
09 ((Nya laughs, Kay looks over and begins laughing. Lou and Max join in))
10 Kay: [Sh-
11 Meg: [Nya put a loop inside a lo:op ((projecting to whole class))
12 David: Ya[:ay
13 Meg: [I'm like- I'm shook]
14 [((Meg mock fans herself with both hands, laughs, and smiles broadly))]
15 ((David laughs, and Lou, Kay, and Nya laugh))
16 Nya: °I'm shook°
17 ((David resumes helping Kay))

c. Nya's code, with a loop within a loop.

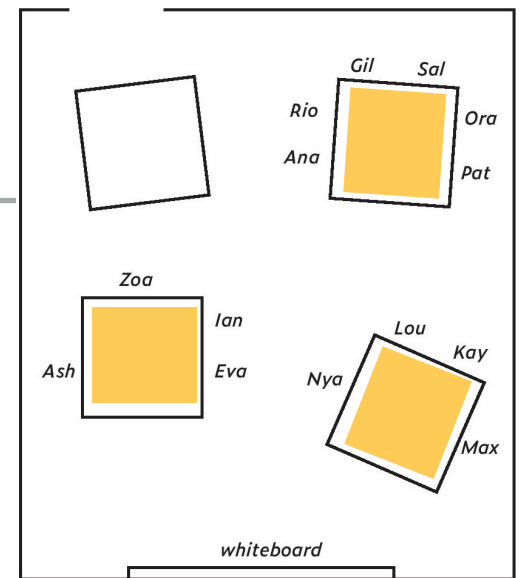
```
repeat(4, ()=>{
  paint('pink')
  forward(4)
  turnRight()
})

forward(1)
turnRight()
forward(1)

repeat(4, ()=>{
  repeat(2, ()=>{
    paint("deepOrange")
    forward(1)
  })
  turnLeft()
})
```



CLOSING



- ▶ By the end of this session, Nya and Max use nested loops; Ash, Eva, Ian, and Kay attempt using the loop; Rio uses copy-and-paste; and Lou, Ora, and Pat write sequences of code commands (without using copy-and-paste)
 - ▶ Only Nya, Max, and Rio complete the challenge
- ▶ Mia invites students to raise their hands to share their solutions with the whole class and calls on Max to start the discussion. After Max shares his nested loop approach, Meg states that students don't "have to do the loop inside the loop, but the important thing to know is that if you ever find that you're copy and pasting code a lot, think about a **better** way to do it -- **or a more efficient** way to do it." Meg then breaks down the components of the loop and collaboratively writes code with a loop on the whiteboard.

DISCUSSION

- ▶ On the other hand, classroom discourse can create inequitable expectations for students to yield to particular visions of sociocomputational norms
- ▶ Inequity was amplified over time *for some students* – they gradually stop entering/do not have access to the conversation floor
- ▶ Inequity also amplified over time *for some students* – they are expected to yield to instructors' preference at this point in time **and** their stances on preferred coding approach ignored, not described, not supported

When you think about the range of emotion that might arise in a classroom (whether around refactoring or not), what do you consider? Stories from your classroom? Ideas as a teacher supporting debugging? **Free-write again. Share.**

Classroom application brainstorm. Notes from out discussion...